

AI Agents Setup Guide

Everything a new team member needs to get Claude Code, Codex, and Pi working with this repo's skills, tools, and optimizations. Follow in order — each section builds on the previous.

What this repo is

A shared library of AI agent skills, tools, standards, and automation scripts. Your projects don't copy from it — they import from it via live @-references. When this repo is updated, your projects pick up changes automatically on the next session.

canon/	← this repo (shared library, clone once)
skills/	← sprint, pdf (catalog); wrapup, capture (deps
– loaded via sprint)	
tools/	← handoff, ticket, tkt.sh (infrastructure –
loaded via sprint, not in catalog)	
standards/	← efficiency (auto-injected into every project
– not in catalog)	
scripts/	← hook automation (handoff, wrapup trigger,
pre-commit)	
guides/	← this file and context-optimization.md
extensions/pi/	← Pi lifecycle extensions
your-project/	← your work repo
CLAUDE.md	← @-imports pointing into canon
AGENTS.md	← skill table + inlined standards for Codex
and Pi	
HANDOFF.md	← session context (auto-managed)

Step 1 — Clone canon

Clone the repo anywhere you like. The scripts self-locate at runtime.

```
git clone https://github.com/sunitghub/canon.git ~/Developer/canon
```

Then set a shell variable for the rest of this guide (substitute your actual clone path):

```
export SKILLS=~/Developer/canon
```

Verify:

```
ls $SKILLS/skills.sh # should exist
```

Step 2 — Install prerequisites

RTK (optional, recommended) — filters verbose CLI output before it hits the AI's token budget. Saves 60–90% of tokens on common operations. Setup detects whether it's installed and wires the hook automatically if so; skills work without it.

```
# macOS
```

```
brew install rtk
```

```
# WSL / Linux
```

```
cargo install rtk
```

```
rtk --version # verify
```

If `rtk gain` fails after install, you may have the wrong package (name collision on crates.io). Use `brew install rtk` — not `cargo install rtk`.

tk — bundled minimal ticket tool, included with canon. No install needed.
`skills add sprint` adds it automatically and offers to put it on your PATH.

Steps 3–5 — Agent setup (automated)

Claude Code only — run the lightweight init command:

```
$SKILLS/skills.sh init
```

Re-run `skills.sh init` any time you move or rename the canon folder — it rewires the hook paths in `~/.claude/settings.json` to the new location.

Claude Code + Codex + Pi — run the full setup script. It is safe to run multiple times:

```
$SKILLS/init-agent.sh
```

It will prompt you to choose an agent (or all), back up any existing config files before modifying them (`.bak` extension), and report what was added vs already present.

You can also run non-interactively:

```
$SKILLS/init-agent.sh claude # Claude Code only
$SKILLS/init-agent.sh codex  # Codex only
$SKILLS/init-agent.sh pi     # Pi only
$SKILLS/init-agent.sh all    # all three
```

What it sets up per agent:

Agent	What gets configured
Claude Code	Handoff + quality hooks merged into ~/.claude/settings.json. RTK hook wired automatically if rtk is installed; skipped with a hint if not.
Codex	RTK instructions via rtk init -g --codex → writes ~/.codex/RTK.md + @reference in ~/.codex/AGENTS.md (skipped if RTK absent).
Pi	Copies extensions/pi/handoff.ts to ~/.pi/agent/extensions/

Manual fallback (if you prefer to inspect before applying):

Claude Code — manual hook setup

```
rtk init -g --auto-patch # RTK native hook (non-interactive)
```

Then either run \$SKILLS/skills.sh init (recommended), or merge manually into ~/.claude/settings.json (replace <SKILLS> with your actual clone path):

```
{
  "hooks": {
    "Stop": [{ "matcher": "", "hooks": [{ "type":
      "command", "command": "<SKILLS>/scripts/auto-
      handoff.sh" }] }],
    "UserPromptSubmit": [{ "matcher": "", "hooks": [{ "type":
      "command", "command": "<SKILLS>/scripts/handoff-
      inject.sh" }] }],
    "PostToolUse": [{ "matcher": "Bash", "hooks": [{ "type":
      "command", "command": "<SKILLS>/scripts/auto-polish-
      trigger.sh" }] }],
    "PreToolUse": [{ "matcher": "Bash", "hooks": [{ "type":
      "command", "command": "<SKILLS>/scripts/pre-commit-
      check.sh" }] }],
  }
}
```

Codex — manual setup

```
rtk init -g --codex --auto-patch
```

Pi — manual setup

```
mkdir -p ~/.pi/agent/extensions
cp $SKILLS/extensions/pi/handoff.ts ~/.pi/agent/extensions/
  handoff.ts
# then /reload in Pi
```

Step 6 — Per-project setup

Run this once per project you want to use skills in.

Register skills

All at once (recommended for new projects):

```
cd /path/to/your-project
$SKILLS/skills.sh addall
```

Or pick individually:

```
cd /path/to/your-project
```

```
# Full dev workflow – sprint covers planning, quality, and tickets
$SKILLS/skills.sh add sprint
```

```
# Optional extras
```

```
$SKILLS/skills.sh add pdf          # PDF read/extract/merge/split
```

addall is idempotent — safe to re-run if new skills have been added to canon since you last registered.

Standards and deps are auto-injected. Every `skills.sh add` call automatically injects the efficiency standard (coding principles, git conventions, token-efficiency rules). Skill deps like `code-reviewer`, `handoff`, and `ticket` are wired silently — not shown in `skills list` or registered explicitly.

Verify registration

```
$SKILLS/skills.sh status          # or: skills.sh --scan /path/to/
                                   your-project
```

This checks registered skills, inline standard freshness, and broken @-import paths. It reports issues and tells you exactly what to run to fix them.

Initialize HANDOFF.md

`handoff` is a dep of `sprint` and loaded automatically. To initialize the file, tell Claude or Codex: “Initialize the handoff file” — it creates `HANDOFF.md` in the project root from the template.

Verification checklist

Run through these to confirm everything is wired up correctly.

RTK (if installed)

```
rtk gain          # should show "No tracking data yet" or savings
                  stats (not an error)
rtk git status    # should run and show compact output
```

Claude Code hooks

```
cat ~/.claude/settings.json # should contain: auto-handoff,
                             handoff-inject, auto-polish-trigger, pre-commit-check
                             (plus rtk hook if RTK is installed)
```

Codex (if RTK installed)

```
cat ~/.codex/AGENTS.md # should contain @RTK.md reference
```

Per-project

```
$SKILLS/skills.sh status # lists registered skills
```

Skill verification

After registering skills, confirm each one is wired up and responding correctly.

Skill	How to verify	Expected response
sprint	"Start a sprint for X"	Sprint brief produced — files, acceptance criteria, blockers listed — awaits approval before any code
pdf	"Extract text from [file].pdf"	Extracted content, or a clear error if no PDF is present
ticket	tkit ls	Empty list or existing tickets — no error

Everything else is automatic. efficiency is injected into every session. capture fires mid-session on any non-obvious discovery. wrapup runs inside sprint complete. handoff and ticket are deps of sprint — wired silently.

Day-to-Day Workflows

How it all fits together

CANON DEV LIFECYCLE

PLAN sprint start	BUILD [write code]	SHIP sprint complete
reads: HANDOFF.md DECISIONS.md sprint files	capture (auto) fires on discovery → HANDOFF.md	wrapup: simplify → code review → security

creates: blueprint.md acceptance.md	## Discoveries → project memory	acceptance check DECISIONS.md update
tkit start ↓ awaits approval		tkit close

Automatic (no user action needed):
 Session start → handoff-inject reads HANDOFF.md into first prompt
 Mid-session → capture writes discoveries to HANDOFF.md
 Session end → auto-handoff snapshots git state to HANDOFF.md

Ad-hoc (outside a sprint):
 /wrapup → quality check on any code without a sprint
 /capture <x> → force-record a discovery the agent missed

A complete session — from idea to shipped

Everything in **bold** is something you type. Everything else happens automatically.

Opening the session

You open Claude Code and type your first message. Before it reaches the agent:

handoff-inject fires — reads HANDOFF.md silently into your first prompt (once per 4-hour window).

The agent wakes up knowing the current project state, prior decisions, and any mid-session discoveries from the last session. You don't have to re-explain context.

You: "Sprint start — add rate limiting to the login endpoint"

The agent:

1. Runs `tkit create "Add rate limiting to login endpoint"` → ticket `t-r4t3`
2. Runs `tkit start t-r4t3`
3. Creates `.tickets/t-r4t3/blueprint.md` and `.tickets/t-r4t3/acceptance.md`
4. Reads `DECISIONS.md` — finds: *"Redis chosen for session state"*
5. Reads `HANDOFF.md` — picks up any open blockers or prior context
6. Produces a sprint brief and waits:

Sprint `t-r4t3` – Add rate limiting to login endpoint

Goal: Block brute-force attempts by tracking failed logins per IP

in Redis.

Files to modify:

auth/views.py - add rate limit check to login endpoint
auth/middleware.py - new @rate_limit decorator

Files to create:

tests/test_rate_limit.py

Acceptance criteria:

- ✓ Login returns 429 after 5 failed attempts from the same IP
- ✓ Counter resets after 15 minutes
- ✓ Rate limit is not bypassable via header manipulation
- ✓ Tests pass

Constraint from DECISIONS.md: use Redis (already a dependency).

Ready to proceed?

You: “Yes”

During the build

The agent writes code. While reading the Redis client config, it notices the connection pool is set to 5 — which could cause rate limit checks to queue under heavy login load.

capture fires automatically: - Appends to HANDOFF.md under ##
Discoveries: “Redis connection pool capped at 5 — rate limit checks may queue under sustained login load. See config/redis.py:12.” - Saves a project memory

You didn’t ask for this. It happened because the agent detected a non-obvious constraint.

You: “Sprint complete”

The agent:

1. **Wrapup pipeline** — runs on all files modified since sprint start:

- code-simplifier — removes a redundant null check in middleware.py, clarifies a variable name
- code-reviewer — flags a missing test for the header-manipulation bypass criterion
- security-review — confirms no injection risk in the Redis key construction

2. **Acceptance check** — reviews acceptance.md:

- ✓ 429 returned after 5 failures
- ✓ Counter resets after 15 minutes
- ✗ No test for header bypass (flagged by reviewer)

- ? Tests pass — uncertain, bypass test missing

Stops. Reports: *“Criterion 3 not met — bypass test is missing. Adding it now.”* Writes the test. Re-checks: all ✓.

3. DECISIONS.md — appends:

Date	Decision	Reason
2026-05-17	Rate limit key uses first IP from X-Forwarded-For	Prevents bypass via appended IPs while preserving proxy compatibility

4. HANDOFF.md — updates ## Next Steps: *“Consider alerting on repeated 429s — out of scope for this sprint.”*

5. Runs `tkl close t-r4t3`

6. Reports: *“Shipped. Rate limiter live on login endpoint. Redis pool cap captured in Discoveries. One decision recorded. Follow-up noted in HANDOFF.”*

Closing the session

You close Claude Code. Before it exits:

auto-handoff fires — appends a snapshot to HANDOFF.md: modified files, recent commits, active tickets.

Next session — or next agent — reads the file and picks up exactly where this one left off.

Sprint reference

sprint start — triggers: *“sprint start”, “start a sprint for X”, “let’s work on X”*

- Creates or identifies the active ticket → `tkl start <id>`
- Creates `.tickets/<id>/blueprint.md` and `acceptance.md`
- Reads DECISIONS.md and HANDOFF.md
- Produces a sprint brief and waits for approval before writing any code

sprint complete — triggers: *“sprint complete”, “approve”, “ship it”*

- Runs wrapup (simplify → review → security) on modified files
- Validates every acceptance criterion — stops if any fail
- Appends new decisions to DECISIONS.md
- Updates HANDOFF.md ## Next Steps
- Runs `tkl close <id>`

DECISIONS.md — repo root, maintained by sprint. Records durable architectural choices with their reasoning. Sprint start reads it; sprint complete writes to it. Separate from HANDOFF.md (which is session state, not permanent record).

Decisions

Date	Decision	Reason
2026-05-17	Amounts stored as integer cents	Avoid float precision bugs

Ticketing with tkt

Sprint manages the ticket lifecycle automatically. Use tkt directly for queries and status checks.

```
tkt ls # list all tickets
tkt ls --status=in_progress # filter by status
tkt show <id> # full ticket detail
tkt reopen <id> # reopen a closed ticket
```

Need dependency tracking, tags, or assignees? Install ticket (brew install ticket) — same .tickets/ format, fully compatible.

Knowledge capture

Fires automatically — no action needed. When the agent finds something non-obvious mid-session (filter rules, row counts, environment gotchas, constraints not visible in the code), it immediately appends to HANDOFF.md ## Discoveries and saves a project memory.

To force-record something the agent missed:

Agent	Trigger
Claude Code	/capture <text>
Codex / Pi	“Capture this” / “Record this in discoveries”

Wrapup — ad-hoc quality check

Runs automatically inside sprint complete. Use it directly only for code written outside the sprint workflow.

/wrapup

Pipeline: code-simplifier → code-reviewer → security-review

Each step skips automatically when it doesn’t apply — the agent states why in one line.

Step	Skipped when
code-simplifier	Single-line change, or docs/config only

Step	Skipped when
code-reviewer	Single-line fix with no design implications
security-review	No auth, DB, user input, API, crypto, or file I/O changed

Staying updated

When this repo is updated with improved skills, standards, or scripts:

```
cd $SKILLS && git pull
```

Hook scripts update immediately — they're called by path, so the new version runs on the next session.

Skill content in `CLAUDE.md` (Claude) is live @-import references — Claude picks up changes automatically on the next session.

Inline standards in `AGENTS.md` (Codex, Pi) are a static copy and need an explicit refresh to pick up changes:

```
$SKILLS/skills.sh refresh /path/to/your-project
```

`refresh` re-registers every skill already in the project, replaces any outdated inline standard blocks in `AGENTS.md`, and heals stale @-import paths — all in one command.

For newly added skills (opt in individually or all at once):

```
$SKILLS/skills.sh addall /path/to/your-project      # register
               any new skills
$SKILLS/skills.sh add <new-skill> /path/to/your-project
               # or just one
```

Check for issues before refreshing:

```
$SKILLS/skills.sh status /path/to/your-project
```

Reports stale paths, outdated inline standards, and broken @-imports — with a one-line fix suggestion when anything is out of date.

Check what skills are available:

```
$SKILLS/skills.sh list
```

How the automation works end-to-end

See [guides/context-optimization.md](#) for a full explanation of the token optimization, session handoff, and quality pipeline — including why each piece is designed the way it is.